

Engenharia de Serviços em Rede

Serviço *Over the Top* para entrega de multimédia

Cláudia Faria^[PG60240], Hugo Marques^[PG60266], and Patrícia Bastos^[PG60287]

Universidade do Minho, Braga, Portugal
{PG60240, PG60266, PG60287}@alunos.uminho.pt
Grupo 38

1 Introdução

O documento aqui apresentado descreve o desenvolvimento de um serviço *Over the Top* (OTT) para entrega de multimédia, no âmbito da unidade curricular de Engenharia de Serviços em Rede.

O objetivo do projeto passa por conceber um protótipo de entrega de conteúdos multimédia com requisitos de tempo real, implementando uma rede de *overlay* aplicacional que optimize o desempenho na transmissão entre servidor e clientes.

2 Arquitetura da Solução

Para a implementação da solução preferiu-se a utilização da linguagem de programação *Python* devido à sua simplicidade, o uso de um protocolo em formato híbrido (TCP e UDP), a construção do *overlay* através de uma abordagem baseada em *flooding* (onde cada servidor inicia *floods* periódicos para propagar informação sobre streams disponíveis e latências), e a seleção de rotas pelos clientes com base em métricas de latência (RTT).

O diagrama da figura 1 apresenta a configuração das camadas da solução desenvolvida.

2.1 Componentes Principais

O sistema é composto por quatro tipos de nós:

Servidor: Responsável por transmitir a *stream* de vídeo, iniciar *floods* periódicos, e manter uma tabela de filhos subscritos. Suporta múltiplas *streams* em simultâneo configuradas através de ficheiros **JSON**.

Router: Responsável por receber *streams* de vídeo, manter tabelas de pais e de potenciais pais, fazer medições de latência (RTT), e selecionar automaticamente o pai com menor latência.

Cliente: Subscrive uma *stream* e utiliza **ffplay** para visualização da *stream* recebida.

Bootstrapper: Responsável pelo arranque dos nós, fornecendo a cada nó a sua lista de vizinhos configurados no ficheiro **config.json**. Atua como serviço de descoberta inicial, permitindo a construção da topologia *overlay*.

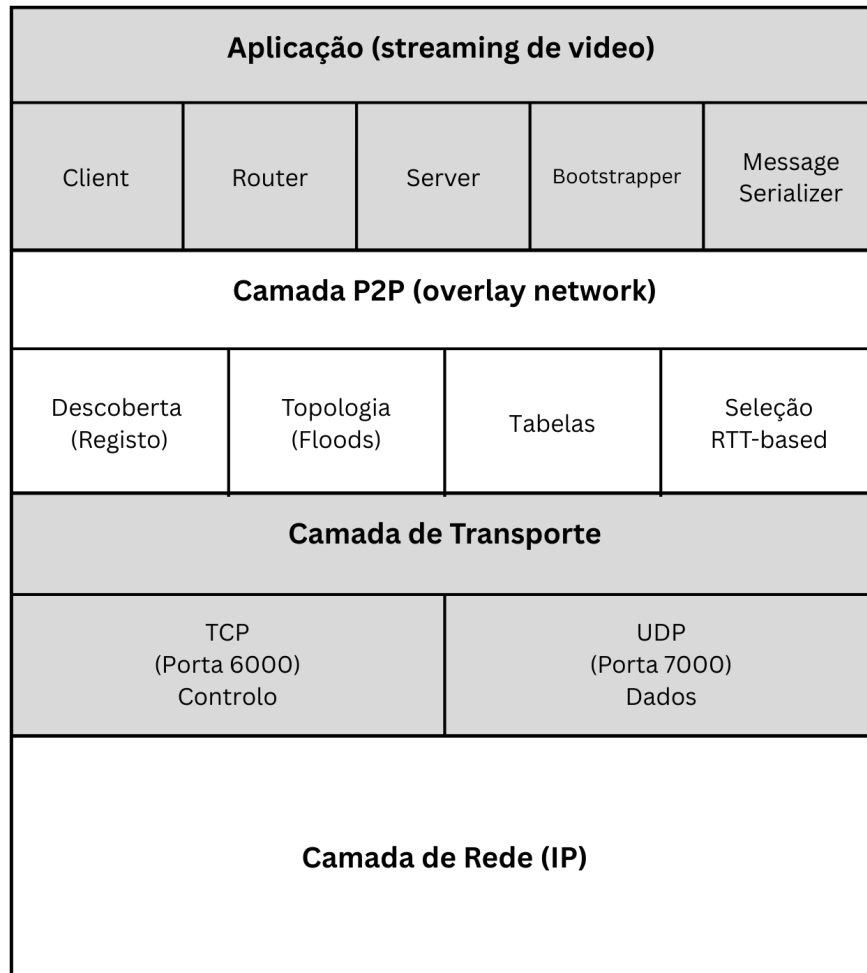


Fig. 1. Camadas.

3 Especificação do protocolo

Como mencionado anteriormente, optou-se pela utilização de um protocolo em formato híbrido que combina TCP e UDP consoante o tipo de comunicação. TCP é usado para mensagens de controlo, sinalização e gestão da rede *overlay* porque se trata de um protocolo *reliable*. Assim é garantido que qualquer mensagem enviada chega sempre ao destino com integridade. UDP é usado para a transmissão de dados multimédia devido à sua rapidez. Apesar de assegurar velocidade à custa de potenciais perdas de pacotes, o grupo considerou ser uma escolha razoável dado o serviço de *streaming* ser considerado tolerante a faltas.

3.1 Formato das Mensagens

Mensagens de Controlo (TCP) As mensagens de controlo seguem o seguinte formato:

`msg_type, node_id, content, msg_id, origin, timestamp`

Table 1. Estrutura das mensagens de controlo TCP

Campo	Descrição
<code>msg_type</code>	Tipo da mensagem. Valores possíveis: TYPE_REGISTRATION: Pedido da lista de vizinhos ao <i>bootstrapper</i> TYPE_SHUTDOWN: Saída de um nó da rede TYPE_PING: Verificação de conectividade (espera TYPE_PONG) TYPE_PONG: Resposta a TYPE_PING TYPE_FLOOD: Propagação de informação por <i>flooding</i> TYPE_RTT_SEND: Pedido de medição de latência TYPE_RTT_REPLY: Resposta a pedido RTT TYPE_JOIN: Subscrição a uma <i>stream</i> TYPE_LEAVE: Cancelamento de subscrição
<code>node_id</code>	Identificador único do nó emissor
<code>content</code>	Informação adicional específica a cada tipo da mensagem: TYPE_REGISTRATION: Identificador do nó e a sua porta TYPE_SHUTDOWN: Lista de vizinhos vivos e IP do nó pai TYPE_FLOOD: ID do <i>flood</i> , latência, <i>streams</i> , <i>timestamp</i> TYPE_JOIN: ID da <i>stream</i> , porta UDP TYPE_LEAVE: Identificador da <i>stream</i>
<code>msg_id</code>	ID único da mensagem (gerado automaticamente caso não especificado)
<code>origin</code>	ID do nó que originou a mensagem (node_id por defeito)
<code>timestamp</code>	<i>Timestamp</i> da criação (gerado automaticamente)

Mensagens de Streaming (UDP) Os dados de *streaming* são enviados através de UDP com o seguinte formato:

`[header_size] [header_json] [video_data]`

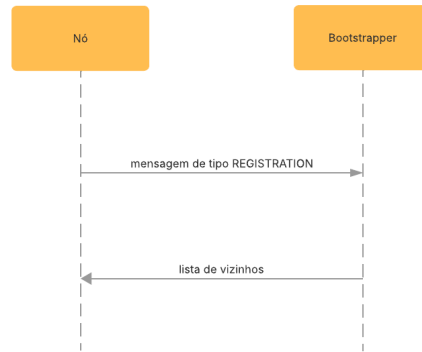
Table 2. Estrutura das mensagens de *streaming* UDP

Campo	Descrição
header_size	Tamanho do <code>header_json</code> em bytes. Permite ao recetor saber quantos <i>bytes</i> deve ler para obter o cabeçalho completo.
header_json	Objeto JSON com metadados do frame. Estrutura: <code>stream_id</code> : Identificador da <i>stream</i> <code>frame_number</code> : Número sequencial do frame na <i>stream</i> <code>timestamp</code> : <i>Timestamp</i> do momento de envio <code>data_size</code> : Tamanho em <i>bytes</i> dos dados de vídeo
video_data	Dados do <i>frame</i> de vídeo em formato JPEG/MJPEG.

3.2 Interações

Registo dos nós

Quando um nó é iniciado, este contacta o *bootstrapper* que responde com a lista de endereços IP dos vizinhos configurados para aquele nó. Esta interação permite a descoberta inicial da topologia, fornecendo a cada nó o conhecimento dos seus vizinhos diretos no *overlay*.

**Fig. 2.** Diagrama de sequência: Registo de um nó

Descoberta de vizinhos através de PING/PONG

O nó envia **ping** aos vizinhos configurados. Os que respondem com **pong** são adicionados à lista de vizinhos vivos. Um nó que recebe **ping** de um vizinho que ainda não consta na lista de vizinhos vivos adiciona-o automaticamente nela.

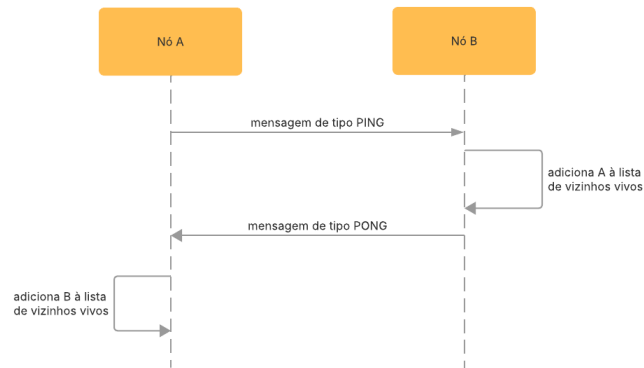


Fig. 3. Diagrama de sequência: Descoberta de vizinhos

Medição RTT

A medição de latência entre nós é realizada através do RTT. Estas medições são essenciais durante o processamento de *floods*, onde cada nó mede o RTT aos vizinhos para os quais irá reencaminhar a mensagem, acumulando a latência total do caminho. As medições são armazenadas e utilizadas para calcular a latência acumulada em cada salto.

Em caso de *timeout* ou falha na medição, o RTT é registado como infinito, sinalizando que o vizinho não está acessível ou o caminho não é viável.

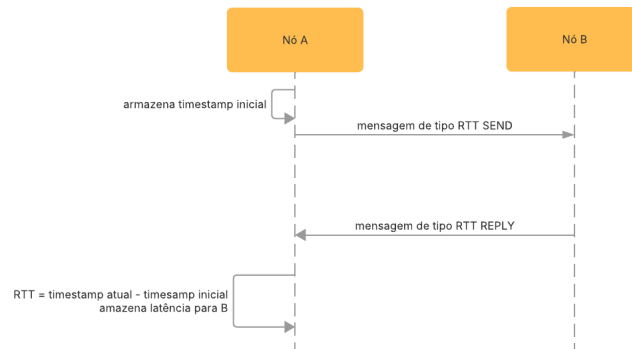


Fig. 4. Diagrama de sequência: medição RTT

Subscrição Stream

O cliente seleciona o pai com menor latência e envia **JOIN**. O pai verifica se o cliente já existe na tabela de filhos (reativa se sim, adiciona se não). Se for o primeiro cliente ativo, propaga o **JOIN** ao seu próprio pai, garantindo que o caminho completo até ao servidor é ativado.

A partir deste momento, os frames de vídeo recebidos via UDP são reen-caminhados para todos os clientes com estado ativo (A) na tabela de filhos para aquela *stream*.

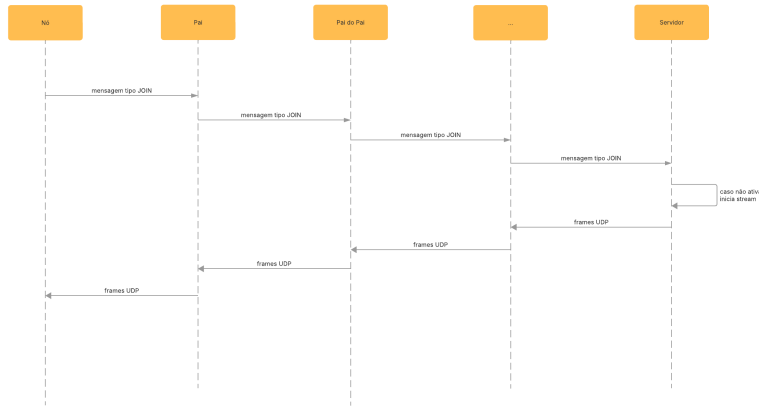


Fig. 5. Diagrama de sequência: subscrição de stream

Dessubscrição Stream

Ao receber **LEAVE**, o nó pai marca o cliente como desativado (D). Se não restarem clientes ativos para aquela *stream*, propaga o **LEAVE** ao seu próprio nó pai, desativando o ramo da árvore de distribuição que deixou de ser necessário. Este mecanismo otimiza o uso de recursos, garantindo que apenas os caminhos efetivamente utilizados permanecem ativos.

Mudança de Pai

A mudança de pai ocorre quando um cliente subscrito a alguma *stream* deteta, através de novos *floods*, um caminho com latência significativamente menor.

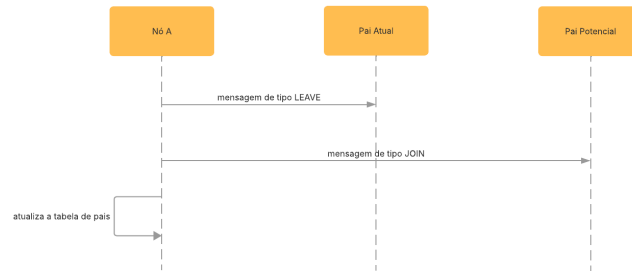


Fig. 6. Diagrama de sequência: mudança de nó pai

Shutdown gracioso

Quando um nó termina, envia **LEAVE** para streams ativas, notifica os seus vizinhos com **SHUTDOWN**, e fecha recursos locais.

Ao receberem a notificação, os vizinhos removem o nó da lista de vizinhos vivos. Se o nó morto era um pai ativo na tabela de pais, o nó herda o pai do nó morto, enviando-lhe um **PING** para estabelecer conectividade. Se a stream se encontra ativa, envia também um **JOIN** para manter a subscrição ativa através do novo caminho.

Este mecanismo de saída graciosa permite que a rede se adapte rapidamente à indisponibilidade de nós, mantendo a integridade das árvores de distribuição. Caso um nó falhe abruptamente sem enviar mensagem, a monitorização periódica através de *floods* permite a deteção indireta da falha (i.e. ausência de *floods* esperados), levando à seleção automática de caminhos alternativos na próxima avaliação.

4 Implementação

4.1 Tabelas de Rotas

Table 3. Tabela de pais.

Campo	Descrição
stream_id	Identificador da stream
node_parent	IP do nó pai que fornece a stream
flood_id	Identificador da mensagem de flood
latency	RTT em ms até ao pai
state	Estado da subscrição

Table 4. Tabela de filhos.

Campo	Descrição
stream_id	Identificador da stream
node_child	IP do nó filho subscrito
state	Estado da subscrição

4.2 Streaming

O servidor lê os frames JPEG, constrói pacotes UDP no formato especificado, e envia para todos os clientes ativos na tabela de filhos para aquela *stream*. O vídeo é transmitido em *loop* contínuo enquanto houver clientes subscritos.

Otimizações implementadas:

- Paragem automática quando não há clientes ativos
- *Buffer* de leitura robusto para lidar com frames fragmentados
- Controlo de taxa (*frame rate*) para sincronização

O **ffplay** é reiniciado automaticamente se crashar, garantindo robustez na reprodução. Estatísticas de receção (*FPS*, *bitrate*, total de frames) são impressas periodicamente para monitorização.

Reencaminhamento (Nós Intermédios)

Nós intermédios (nem servidores nem clientes finais) reencaminham *frames* UDP recebidos para todos os filhos ativos na tabela de filhos para aquela *stream*.

5 Testes e Resultados

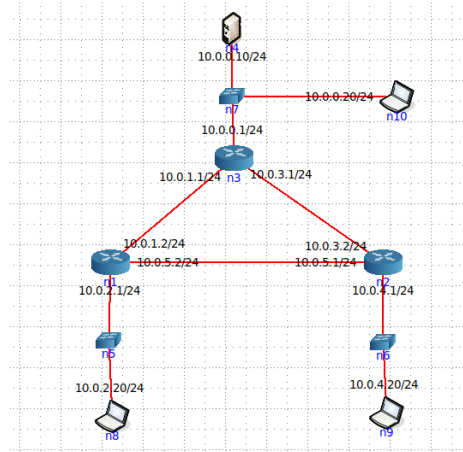


Fig. 7. Overlay de teste simples

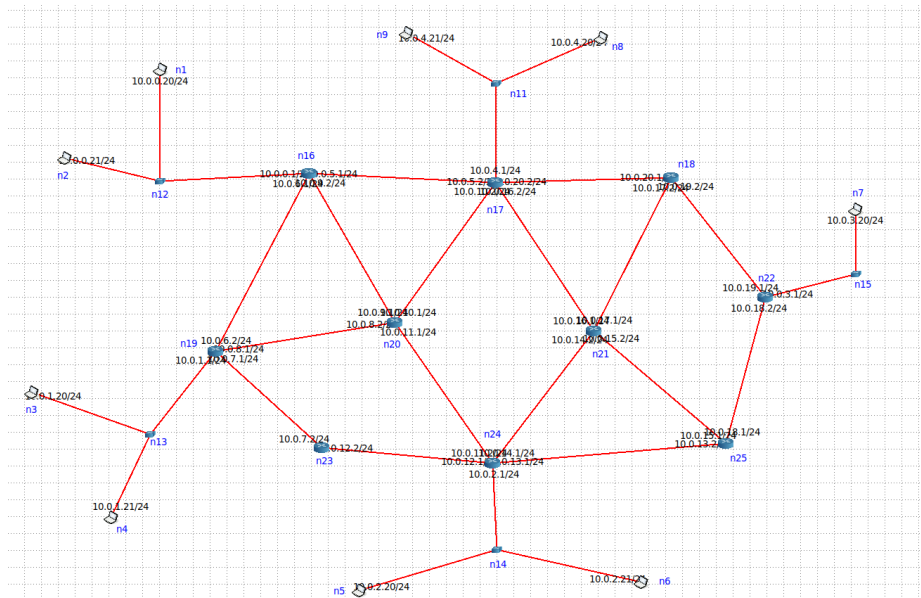


Fig. 8. Overlay de teste complexo

5.1 Descoberta de vizinhos

Usando o *overlay* simples, foram iniciados os *routers* 1, 2 e 3. Verifica-se na imagem que todos os nós receberam corretamente os seus vizinhos do *bootstrapper* e atualizaram os seus vizinhos à medida que recebeu informação da sua chegada.

```

=====
Node n3 completely initialized
TCP Port: 5000
UDP Port: 7000
Alive neighbors: []
Child table status:

Child Table: EMPTY

=====
Node n3 Neighbor 10.0.1.2 is now alive!
Updated alive neighbors: ['10.0.1.2']

=====
Node n3 Neighbor 10.0.3.2 is now alive!
Updated alive neighbors: ['10.0.1.2', '10.0.3.2']

=====
[]

=====
root@rd:/tmp/putone.41355/n4.conf# su - core
core@rd:~$ python3 bootstrapper.py

=====
Successfully loaded config from config.json
Bootstrapper running on 10.0.0.25000
Using config file: config.json
Bootstrapper started, waiting for connections...
EBRC: Node n3 registering from 10.0.0.15000
EBRC: Found 3 neighbors for n3: ['10.0.0.20', '10.0.1.2', '10.0.3.2']
EBRC: Node n1 registering from 10.0.1.25000
EBRC: Found 3 neighbors for n1: ['10.0.1.1', '10.0.5.1', '10.0.2.20']
EBRC: Node n2 registering from 10.0.3.25000
EBRC: Found 3 neighbors for n2: ['10.0.3.1', '10.0.5.2', '10.0.4.20']

=====

core@rd:~$ python3 node.py n1 10.0.0.10
No n1 incluido na porta TCP 5000 e UDP 7000
Received 3 neighbors from bootstrapper: ['10.0.1.1', '10.0.5.1', '10.0.2.20']
Checking 3 neighbors...

=====
Node n1 Neighbor 10.0.1.1 is now alive!
Updated alive neighbors: ['10.0.1.1']

=====
Neighbor 10.0.5.1 did not respond: [Errno 111] Connection refused
Neighbor 10.0.2.20 did not respond: [Errno 111] Connection refused
Found 1 alive neighbors: ['10.0.1.1']
Started UDP listener on port 7000

=====
Node n1 completely initialized
TCP Port: 5000
UDP Port: 7000
Alive neighbors: ['10.0.1.1']
Child table status:

Child Table: EMPTY

=====
Node n1 Neighbor 10.0.5.1 is now alive!
Updated alive neighbors: ['10.0.1.1', '10.0.5.1']

=====

core@rd:~$ python3 node.py n2 10.0.0.10
No n2 incluido na porta TCP 5000 e UDP 7000
Received 3 neighbors from bootstrapper: ['10.0.3.1', '10.0.5.2', '10.0.4.20']
Checking 3 neighbors...

=====
Node n2 Neighbor 10.0.3.1 is now alive!
Updated alive neighbors: ['10.0.3.1']

=====
Node n2 Neighbor 10.0.5.2 is now alive!
Updated alive neighbors: ['10.0.3.1', '10.0.5.2']

=====
Neighbor 10.0.4.20 did not respond: [Errno 111] Connection refused
Found 2 alive neighbors: ['10.0.3.1', '10.0.5.2']
Started UDP listener on port 7000

=====
Node n2 completely initialized
TCP Port: 5000
UDP Port: 7000
Alive neighbors: ['10.0.3.1', '10.0.5.2']
Child table status:

Child Table: EMPTY

=====

```

Fig. 9. Descoberta de vizinhos

5.2 Geração de rotas

Usando o *overlay* simples, com todos os *routers* e servidor ativos, verificam-se os *logs* que identificam a atualização da tabela Pai após a chegada de um *flood*. Note-se que é também visível o crescimento de latência ao longo do percurso. O estado da *stream* encontra-se desativado, uma vez que ainda não existe cliente a pedir a *stream*.

```

vcmd
Received RTT request from 10.0.0.20
Sent RTT reply to 10.0.0.20

Node n3 received FLOOD
=====
Flood ID: n0.1
From: 10.0.0.20
Latency: 0.4633829296875 ms
Stream: ['movie1']
First flood received from origin n0 for stream movie1.
Adding parent: 10.0.0.20 with latency 0.67 ms
=====
PARENT TABLE - Node n3
=====
Parent IP      Flood ID      Stream ID      State      Latency
(ms)
-----
10.0.0.20      n0.1          movie1         D          0.67

vcmd
[Flood #1]
Server n0 INITIATING PERIODIC FLOOD
=====
Available stream: ['movie1']
Phase 1: Measuring RTT to 1 neighbors...
Measuring RTT to 1 neighbors: ['10.0.0.1']
Starting RTT measurement to 10.0.0.1...
RTT to 10.0.0.1: 0.87 ms
Phase 2: Sending flood to neighbors...
To 10.0.0.1: RTT 0.87 ms
Flood initiated successfully to 1/1 neighbors
Flood ID: n0.1
=====
^C
Stopping server n0
Server n0 stopped
corba@luc:~$

vcmd
Flood ID: n0.1
From: 10.0.0.1
Latency: 14.569441131918 ms
Stream: ['movie1']
Alternative parent found for stream movie1.
Current best: 3.14 ms via 10.0.1.1
Alternative: 14.57 ms via 10.0.1.1
=====
PARENT TABLE - Node n1
=====
Parent IP      Flood ID      Stream ID      State      Latency
(ms)
-----
10.0.0.1        n0.1          movie1         D          3.14

Phase 1: Measuring RTT to our neighbors...
Measuring RTT to 1 neighbors: ['10.0.5.1']
Starting RTT measurement to 10.0.5.1...
RTT to 10.0.5.1: 2.12 ms
Phase 2: Forwarding to 1 neighbors...
To 10.0.5.1: RTT 2.12 ms, total: 16.69 ms
Flood forwarded to 1/1 neighbors

vcmd
Node n2 received FLOOD
=====
Flood ID: n0.1
From: 10.0.5.2
Latency: 18.68115234375 ms
Stream: ['movie1']
Alternative parent found for stream movie1.
Current best: 3.07 ms via 10.0.3.1
Alternative: 18.68 ms via 10.0.5.2
=====
PARENT TABLE - Node n2
=====
Parent IP      Flood ID      Stream ID      State      Latency
(ms)
-----
10.0.3.1        n0.1          movie1         D          3.07

Phase 1: Measuring RTT to our neighbors...
Excluding best parent 10.0.3.1 for stream movie1
No neighbors to measure RTT for
No neighbors to forward to
=====

```

Fig. 10. Geração de rotas

5.3 Streaming básico

Ao testar um *streaming* básico (um servidor com uma *stream*) no *overlay* simples, verificamos o correto funcionamento da funcionalidade de *streaming*.

```

vcmd
[STREAMING] Frame 800 | FPS(avg): 29.6 | Data: 8.43 MB
[STREAMING] Frame 850 | FPS(avg): 29.6 | Data: 8.99 MB
Received RTT request from 10.0.0.1
Sent RTT reply to 10.0.0.1

Client n8 received FLOOD
=====
From: 10.0.0.1
Latency: 5.72413159564944 ms
Contains our stream: movie1
Received newer flood 4 from current parent
Preserving active state.
New latency: 5.72 ms
=====
PARENT TABLE - Node n8
=====
Parent IP      Flood ID      Stream ID      State      Latency
(ms)
-----
10.0.0.1        n0.4          movie1         A          5.72

Received RTT request from 10.0.0.1
Sent RTT reply to 10.0.0.1

Client n8 received FLOOD
=====
From: 10.0.0.1
Latency: 7.09598376201788 ms
Contains our stream: movie1
Alternative parent found for stream movie1.
Current best: 5.72 ms via 10.0.0.1
Alternative: 7.06 ms via 10.0.0.1
=====
PARENT TABLE - Node n8
=====
Parent IP      Flood ID      Stream ID      State      Latency
(ms)
-----
10.0.0.1        n0.4          movie1         A          5.72

[STREAMING] Frame 900 | FPS(avg): 29.8 | Data: 7.47 MB
[STREAMING] Frame 950 | FPS(avg): 29.6 | Data: 7.80 MB
[CLIENT] Detected loop wrap X2 (frame 499 -> 0)
[STREAMING] Frame 1000 | FPS(avg): 28.7 | Data: 8.11 MB

```

Fig. 11. Streaming

Ao verificar os *logs* do *router* 3, é possível confirmar que a sua tabela Pai foi atualizada para A (Ativo) para o *stream* em questão. No servidor 10, podemos verificar que a sua tabela Filho também foi corretamente atualizada para a *stream*, e que o envio dos pacotes decorre normalmente.

The image shows two terminal windows. The left window, titled 'vcmd', displays logs for 'Phase 1: Measuring RTT to our neighbors...' and 'Phase 2: Forwarding to 1 neighbors...'. It includes a table for the 'PARENT TABLE - Node n3' with columns: Parent IP (n3), Flood ID, Stream ID, State, and Latency. The table shows a single entry for Parent IP 10.0.0.20, Flood ID n10.4, Stream ID movie1, State A, and Latency 2.01. The right window, titled 'vcmd', displays logs for '[STREAM movie1] ===== VIDEO LOOP #1 =====' and '[STREAM movie1] Sent frame 100 to 1 children'. It includes a table for the 'CHILD TABLE - Node n10' with columns: Child IP, Stream ID, and State. The table shows a single entry for Child IP 10.0.0.1, Stream ID movie1, and State A. Below the table, it states 'Total children: 1', 'Active children: 1', and 'Active streams: movie1: 1'.

Fig. 12. Atualização de Tabelas Pai e Filho

5.4 Streaming avançado

Primeiro, na *overlay* complexa, foram testados dois fluxos de *streams* diferentes a partir do mesmo servidor.

The image shows three terminal windows and two video player windows. The leftmost terminal window, titled 'vcmd', displays logs for 'STREAM movie1' and 'STREAM movie2', showing frame sending to children and reaching the end of the loop. The middle terminal window, titled 'vcmd', displays logs for 'Phase 1: Measuring RTT to 1 neighbors...' and 'Phase 2: Sending Flood to neighbors...'. It includes a table for the 'PARENT TABLE - Node n5' with columns: Parent IP (n5), Flood ID, Stream ID, State, and Latency. The table shows two entries: Parent IP 10.0.2.1, Flood ID n1.2, Stream ID movie1, State A, Latency 33.16; and Parent IP 10.0.2.1, Flood ID n1.2, Stream ID movie2, State B, Latency 33.16. The rightmost terminal window, titled 'vcmd', displays logs for 'STREAM movie2' and 'STREAM movie1', showing frame sending to children. The two video player windows on the right show a scene with a person in a field, representing the video streams being tested.

Fig. 13. Dois fluxos

Depois, foram testados dois servidores diferentes com duas *streams* diferentes. Para isso, foram iniciados os *routers* 16, 17, 18, 20, e 24. Posteriormente, foram iniciados os servidores 3 e 8, com diferentes *streams* disponíveis, e os clientes 1 e 5, pedindo duas *streams* diferentes.

```

root@n3:~/tmp/pycore.44473/n3.conf# su - core
core@n3:~$ python3 server.py n8 10.0.3.20 ./server_config.json
N8 n3 iniciado na porta TCP 6000 e UDP 7000
Loaded stream: movie1 -> ./movie.Mjpeg (MUJEG)

*****
STREAMS LOADED FROM CONFIG
Config file: ./server_config.json
Streams loaded: 1
*****

Received 1 neighbors from bootstrapper: ['10.0.1.1']
Checking 1 neighbors...

*****
Node n3: Neighbor 10.0.1.1 is now alive!
Updated alive neighbors: ['10.0.1.1']
*****

Found 1 alive neighbors: ['10.0.1.1']
Started UDP listener on port 7000

*****
Server n3 completely initialized
TCP Port: 6000
UDP Port: 7000
Alive neighbors: ['10.0.1.1']
Available streams:
- movie1: ./movie.Mjpeg (MUJEG)
*****

root@n8:~/tmp/pycore.44473/n8.conf# su - core
core@n8:~$ python3 server.py n8 10.0.3.20 ./server_config4.json
N8 n8 iniciado na porta TCP 6000 e UDP 7000
Loaded stream: movie2 -> ./movie.Mjpeg (MUJEG)

*****
STREAMS LOADED FROM CONFIG
Config file: ./server_config4.json
Streams loaded: 1
*****

Received 1 neighbors from bootstrapper: ['10.0.4.1']
Checking 1 neighbors...

*****
Node n8: Neighbor 10.0.4.1 is now alive!
Updated alive neighbors: ['10.0.4.1']
*****

Found 1 alive neighbors: ['10.0.4.1']
Started UDP listener on port 7000

*****
Server n8 completely initialized
TCP Port: 6000
UDP Port: 7000
Alive neighbors: ['10.0.4.1']
Available streams:
- movie2: ./movie.Mjpeg (MUJEG)
*****

```

Fig. 14. Dois servidores

[illegible]

Fig. 15. Dois servidores, duas *streams*

Novamente, verificou-se o correto envio dos pacotes e reprodução da *stream*.

5.5 Saída graciosa dos nós

Na *overlay* simples, foram iniciados todos os *routers* a receber *floods* do servidor. O *router* 3 foi desligado, avisando os seus vizinhos da sua saída e fornecendo-lhes a sua lista de vizinhos. É possível verificar na imagem que o nó propaga o **LEAVE** ao seu pai e informa os vizinhos, que reconhecem a sua saída e herdaram o pai do seu pai, enviando um **JOIN**.

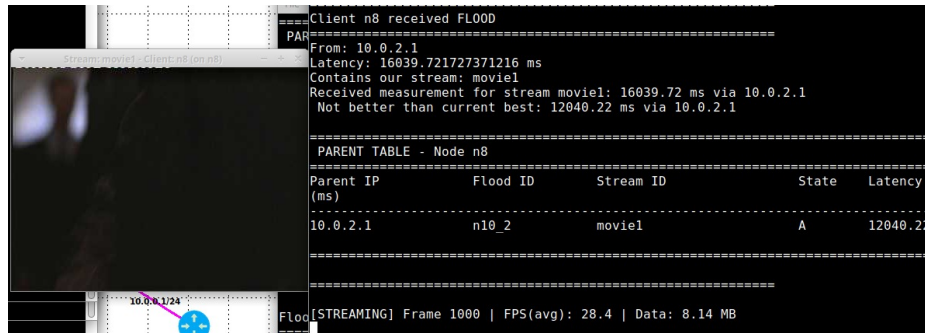


Fig. 18. 1 milhão de microssegundos de latência

Verificou-se que o vídeo continua a ser reproduzido com qualidade, apesar da alta latência.

6 Conclusões e Trabalho Futuro

Conclui-se que o presente trabalho foi essencial para a consolidação do conhecimento sobre *streaming* de multimídia e protocolos associados. Cada escolha implicou trade-offs, especialmente na velocidade de envio de pacotes e mensagens e a sua garantia de integridade.

Em particular, se houvesse disponibilidade para tal, funcionalidades como a saída graciosa de um nó seriam ainda mais optimizadas. Poderiam ter sido usadas métricas de geração de rotas mais complexas, incluindo parâmetros como número de saltos e perdas de pacotes, como inicialmente planeado.